Marmopedras

Sistema de Controle de Estoque

Documentação Técnica — versão atualizada

Projeto Marmopedras

11 de novembro de 2026

Resumo

Este documento descreve o sistema Marmopedras, uma aplicação web para controle de estoque de materiais de uma marmoraria. O front-end é desenvolvido em Next.js/React e o back-end é executado no Back4App como Cloud Code (Parse Server), concentrado em um único arquivo JavaScript. Esta revisão atualiza a documentação inicial (7 de outubro de 2026) com as mudanças do back-end: estoque em chapas inteiras, verificação de e-mail por código, recuperação de senha, planta dos depósitos e envio de e-mails transacionais. Também inclui a suíte de testes automatizados (178 testes em Jest). São apresentados a arquitetura, o modelo de dados, as funções do servidor, as regras de negócio, a integração entre as camadas, a implantação, os testes e as limitações conhecidas.

Sumário

Histórico de revisões **3**

1 Introdução **3**

2 Visão geral da arquitetura **3**
2.1 Tecnologias 4

3 Funcionalidades **4**

4 Estrutura do projeto **5**

5 Modelo de dados **6**
5.1 `_User` 6
5.2 Classes de estoque 6
5.3 `Movimentacao` 6
5.4 `EmailToken` 7
5.5 `PlantaDeposito` 7
5.6 `Lead` 7

6 Cloud Functions **7**
6.1 Conta, verificação e e-mail . . . 7
6.2 Estoque . . . 8
6.3 Consulta . . . 9
6.4 Planta dos depósitos . . . 9
6.5 Leads . . . 9
6.6 Hooks . . . 10
6.7 Tratamento de erros . . . 10

7 Regras de negócio **10**
7.1 Material . . . 10
7.2 Status do material . . . 10
7.3 Entrada . . . 11
7.4 Saída . . . 11
7.5 Transferência . . . 11
7.6 Histórico . . . 11
7.7 Verificação de e-mail . . . 11
7.8 Recuperação de senha . . . 11
7.9 Planta dos depósitos . . . 12

8 Autenticação e segurança **12**

9 Integração do front-end **13**

10 Instalação e implantação **13**
10.1 Back-end (Back4App) . . . 13
10.2 Front-end . . . 14
10.3 Problemas comuns . . . 14

11 Testes automatizados **15**
11.1 Estratégia . . . 15
11.2 Cobertura funcional . . . 15
11.3 Execução e resultado . . . 16
11.4 Limites da suíte . . . 16

12 Conclusão **16**

Histórico de revisões

Data	Versão	Descrição
07/10/2026	1.0	Documentação inicial do sistema.
11/11/2026	2.0	Estoque em chapas, verificação de e-mail, recuperação de senha, planta dos depósitos, e-mail transacional, suíte de testes automatizados e atualização das limitações.

Nota. As seções que descrevem o front-end (telas, estrutura de pastas do Next.js e `api.js`) foram mantidas da versão 1.0, pois o código do front-end não fez parte desta revisão. Elas devem ser conferidas contra o repositório atual.

1 Introdução

A Marmopedras comercializa pedras naturais e sintéticas, em chapas, para diferentes obras. O controle desses materiais, distribuídos em vários depósitos e retirados para diferentes obras, exige registro confiável de cada movimentação e visão rápida do saldo disponível.

O sistema tem como objetivos:

- registrar entradas, saídas e transferências de materiais, sempre em chapas inteiras;
- mostrar o saldo por material e por depósito, com alerta de estoque baixo ou crítico;
- manter o histórico auditável de movimentações, com responsável, data e hora;
- oferecer um pré-atendimento automatizado ao cliente (chatbot de WhatsApp) que gera leads para a equipe de vendas;
- restringir o acesso a usuários autenticados, com e-mail verificado por código e recuperação de senha;
- permitir o desenho da planta dos depósitos, com a posição dos cavaletes.

2 Visão geral da arquitetura

O sistema segue uma arquitetura em duas camadas. O navegador executa a aplicação Next.js e se comunica com o Back4App por meio de chamadas HTTPS às Cloud Functions, que concentram todas as regras de negócio e o acesso ao banco de dados. Para os e-mails de verificação, recuperação de senha e aviso ao administrador, o Cloud Code chama a API HTTP de um provedor externo (Brevo ou Resend).

Cada requisição leva os cabeçalhos `Application-Id`, `JavaScript-Key` e, após o login, `Session-Token`.

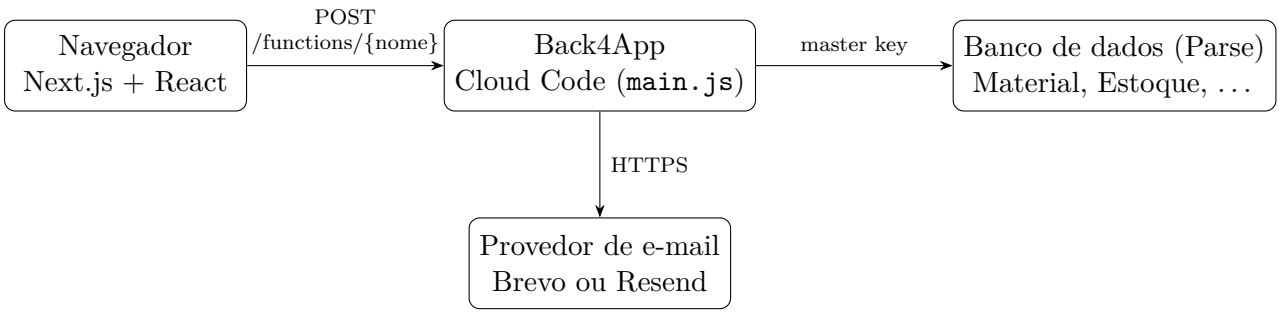


Figura 1: Arquitetura do sistema.

2.1 Tecnologias

Camada	Tecnologia
Front-end	Next.js 15, React 19, JavaScript/JSX
Estilização	Tailwind CSS v4 e CSS global
Back-end	Back4App Cloud Code (Parse Server), JavaScript puro
Banco de dados	Banco gerenciado do Back4App (Parse)
Autenticação	`Parse.User` com token de sessão e e-mail verificado por código
E-mail transacional	API HTTP do Brevo (padrão) ou do Resend
Testes	Jest 30, com implementação do Parse em memória (Node.js 18 ou superior)

3 Funcionalidades

A Tabela 1 relaciona cada funcionalidade do sistema à Cloud Function utilizada.

Tabela 1: Funcionalidades e funções do servidor.

Funcionalidade	Descrição	Função(ões)
Cadastro e login	Criação de conta e autenticação	`register`, `login`, `me`
Verificação de e-mail	Confirmação da conta por código de 6 dígitos	`verificarEmail`, `reenviarVerificacao`
Recuperação de senha	Troca de senha por código enviado por e-mail	`solicitarRecuperacaoSenha`, `verificarCodigoReset`, `redefinirSenha`
Painel Geral	Indicadores do dia, últimas movimentações e materiais críticos	`painel`
Pesquisar Materiais	Busca por nome/código, categoria e status	`listarMateriais`, `categorias`, `criarMaterial`
Registrar Entrada	Adiciona chapas ao estoque de um depósito, podendo cadastrar o material na hora	`registrarEntrada`, `ultimasMovimentacoes`

Funcionalidade	Descrição	Função(ões)
Registrar Saída	Retira chapas para uma obra	`registrarSaida`
Transferência	Move chapas entre depósitos	`registrarTransferencia`
Distribuição	Estoque por depósito	`distribuicao`
Histórico	Movimentações com filtros e paginação	`listarMovimentacoes`
Planta dos depósitos	Cadastro de depósitos e posição dos cavaletes	`carregarPlanta`, `salvarPlanta`, `criarDepositoPlanta`, `excluirDepositoPlanta`
Chatbot WhatsApp	Fluxo de pré-atendimento que grava o lead	`salvarLead`, `listarLeads`

4 Estrutura do projeto

O projeto é dividido em dois repositórios.

Front-end (conforme versão 1.0)

```
app/
  api.js              cliente das Cloud Functions
  App.jsx             layout, navegacao e Painel Geral
  components/Icons.jsx
  pages/
    Auth.jsx
    ChatbotWhatsApp.jsx
    DistribuicaoEstoque.jsx
    HistoricoMovimentacoes.jsx
    PesquisarMateriais.jsx
    RegistrarEntrada.jsx
    RegistrarSaida.jsx
    Transferencia.jsx
  globals.css, layout.js, page.js
public/               imagens
.env.example          modelo das variaveis de ambiente
```

Back-end (`backend-marmopedras`)

```
main.js               todas as funcoes e hooks do servidor
package.json          jest e script "test"
test/
  fakeParse.js        Parse em memoria
  ctx.js              carrega main.js e expoe helpers
  auth.test.js        conta, e-mail, senha, login, hooks
  estoque.test.js     materiais, movimentacoes, painel, leads
  planta.test.js      planta dos depositos
```

Na implantação, o `main.js` é enviado ao Back4App como `cloud/main.js`.

5 Modelo de dados

As classes são criadas automaticamente pelo Parse no primeiro registro salvo. As tabelas seguintes descrevem os campos de cada uma.

5.1 `_User`

Campo	Descrição
`username`, `email`	e-mail do usuário (em minúsculas)
`password`	senha (armazenada com hash pelo Parse)
`nome`, `telefone`	dados de contato
`cargo`	padrão "Funcionário"
`emailVerificado`	`false` até a confirmação do código; contas sem esse campo são tratadas como verificadas

5.2 Classes de estoque

Classe	Campos
`Deposito`	`id_` (A–C), `nome`, `capacidade` (opcional; padrão 0 quando ausente)
`Material`	`cod` (MAT-001…), `nome`, `categoria`, `tipo` (pode ser vazio), `unidade` (sempre "chapas"), `minimo`
`Estoque`	`cod`, `deposito`, `qtd`: saldo de um material em um depósito

Os depósitos válidos para movimentação são fixos: `Depósito A`, `Depósito B` e `Depósito C`.

5.3 `Movimentacao`

Registra cada operação de estoque.

Campo	Descrição
`tipo`	Entrada, Saída ou Transferência
`cod`, `material`, `unidade`	identificação do material
`quantidade`, `qtd`	valor numérico e texto formatado (`+80 chapas`, `-20 chapas`, `10 chapas`)
`local`	depósito, "origem → obra" ou "origem → destino"
`obra`	destino (somente saídas)
`de`, `para`	depósitos (somente transferências)
`nota`, `obs`	nota fiscal (padrão "—") e observações
`responsavel`	usuário (ou nome informado na saída)
`data`, `hora`	AAAA-MM-DD e HH:MM, fuso `America/Recife`

Nas respostas de listagem, cada movimentação é enriquecida com `codigo`, `categoria` e `tipo_material`, obtidos do cadastro do material.

5.4 `EmailToken`

Guarda os códigos de verificação e de recuperação de senha. A classe tem ACL vazia (acessível apenas pela master key).

Campo	Descrição
`userId`	usuário dono do código
`tipo`	`verificacao` ou `reset`
`hash`	HMAC-SHA256 do código (o código em si nunca é gravado)
`expiraEm`	validade: 30 min (verificação) ou 15 min (recuperação)
`usado`, `tentativas`	controle de uso único e de tentativas erradas

5.5 `PlantaDeposito`

Um registro por depósito da planta (ACL vazia).

Campo	Descrição
`nome`, `ordem`	nome do depósito e posição na lista
`largura`, `altura`	tamanho da planta
`cavaletes`	lista de objetos com `id`, `nomeCustomizado`, `x`, `y`, `orientacao` (`horizontal` ou `vertical`) e `anotacoes`

5.6 `Lead`

Pré-atendimento do chatbot: `nome`, `localizacao`, `tipoObra`, `projetoMedidas`, `material`, `corEstilo`, `telefone` e `status` (inicia como `novo`).

6 Cloud Functions

Todas as funções estão em `main.js`. As marcadas como *logado* exigem token de sessão válido de um usuário com e-mail verificado; sem token retornam o código Parse 209.

6.1 Conta, verificação e e-mail

Função	Acesso	Parâmetros	Retorno
`register`	público	nome, email, telefone, cargo, senha	`precisaVerificar`, `email`, `emailEnviado` (e `reenviado` para cadastro pendente)
`verificarEmail`	público	email, codigo	`verificado`
`Reenviar`	público	email	`ok`

Função	Acesso	Parâmetros	Retorno
`login`	público	email, senha	token e usuário
`me`	logado	—	usuário atual
`solicitarSenha`	público	email	`ok` (sempre)
`verificarReset`	público	email, codigo	`ok`, `valido`
`redefinirSenha`	público	email, codigo, novaSenha	`ok`
`testarEmail`	master key	para (opcional)	`ok` e provedor, ou diagnóstico da configuração
`seed`	master key	—	mensagem informando que a carga demonstrativa está desativada (também registrada como job)

Algumas funções têm nomes alternativos, registrados para facilitar a integração com o front-end:

Função	Aliases
`verificarEmail`	`verify-email`, `verifyEmail`
`reenviarVerificacao`	`resend-verification`, `resend-code`, `resendVerification`
`solicitarRecuperacaoSenha`	`forgot-password`, `forgotPassword`, `resend-reset-code`
`verificarCodigoReset`	`verify-reset-code`, `verifyResetCode`
`redefinirSenha`	`reset-password`, `resetPassword`

Os parâmetros também aceitam variações: `codigo` pode vir como `code` ou `token`; `novaSenha` pode vir como `senha`, `password`, `newPassword` ou `nova_senha`.

6.2 Estoque

Função	Acesso	Parâmetros	Retorno
`listarMateriais`	logado	q, categoria, status	lista com estoque total, saldo por depósito, depósito principal e status
`categorias`	logado	—	"Todos", as cinco categorias fixas e eventuais extras
`criarMaterial`	logado	nome, categoria, tipo, minimo	material criado (código automático)
`registrarEntrada`	logado	material ou codigo, novoMaterial, categoria, tipo, quantidade, local, nota, data, obs	movimentação (com `materialNovo`)

Função	Acesso	Parâmetros	Retorno
`registrarSaida`	logado	material ou codigo, quantidade, obra, data, responsavel, obs, local	movimentação
`registrarTransferencia`	logado	material ou codigo, quantidade, origem, destino, obs	movimentação

6.3 Consulta

Função	Acesso	Parâmetros	Retorno
`painel`	logado	—	cartões do dia, últimas cinco movimentações e até três materiais críticos
`listarMovimentacoes`	logado	tipo, inicio, fim, busca, page, limit	itens, total, páginas e resumo por tipo
`ultimasMovimentacoes`	logado	tipo (`entradas`, `saidas`, `transferencias`)	últimas cinco
`distribuicao`	logado	—	depósitos A, B e C com seus materiais e ocupação

6.4 Planta dos depósitos

Função	Acesso	Parâmetros	Retorno
`carregarPlanta`	logado	—	`depositos`, `plantas` e `tamanhos`; cria A, B e C na primeira chamada
`salvarPlanta`	logado	depositos, plantas, tamanhos	`ok`, contagens e `salvoEm`
`criarDepositoPlanta`	logado	nome	depósito criado
`excluirDepositoPlanta`	logado	nome	`ok` e depósitos restantes

6.5 Leads

Função	Acesso	Parâmetros	Retorno
`salvarLead`	público	dados do chatbot (nome obrigatório)	lead salvo
`listarLeads`	logado	—	até 200 leads, mais recentes primeiro

6.6 Hooks

Hook	Comportamento
`beforeSave` (`_User`)	Todo usuário novo nasce com `emailVerificado = false`. Fora da master key, é proibido alterar esse campo em usuário existente.
`beforeLogin`	Bloqueia o login de conta com e-mail não verificado (código `EMAIL_NAO_VERIFICADO`).
`afterSave` (`_User`)	Em novo cadastro, avisa o e-mail definido em `ADMIN_EMAIL`, com os dados escapados contra HTML. Falha no envio é apenas registrada em log.

6.7 Tratamento de erros

Erros de validação são lançados com o código Parse 141 e uma mensagem em JSON, interpretada pelo front-end:

```
{ "erro": "Estoque insuficiente",
  "campos": { "quantidade": "Estoque insuficiente (disponível: 18 chapas)" } }
```

As chaves de `campos` coincidem com os nomes dos campos dos formulários, de modo que a mensagem aparece junto ao campo correspondente. Quando existe, o campo `codigo` identifica uma situação específica (por exemplo, `EMAIL_NAO_VERIFICADO`). A ausência de login retorna o código 209.

7 Regras de negócio

7.1 Material

- A unidade é sempre **chapas**, e toda quantidade (movimentação ou estoque mínimo) é um número inteiro. Quantidades de movimentação devem ser maiores que zero; o mínimo pode ser zero.
- Categorias válidas: Granitos, Mármores, Quartzitos, Sintéticos e Dolomíticos. O tipo, opcional, pode ser Escovado, Acetinado ou Polido.
- O código é gerado em sequência (`MAT-001`, `MAT-002`...).
- Não podem existir dois materiais com o mesmo nome, categoria e tipo.
- Materiais antigos com unidade diferente de "chapas" ficam bloqueados para movimentação até que a unidade seja corrigida no Back4App.

7.2 Status do material

Com E sendo o estoque total (soma dos depósitos A, B e C) e M o estoque mínimo:

$$\text{status} = \begin{cases} \text{ok}, & M \leq 0 \text{ ou } E \geq M \\ \text{crítico}, & E < 0{,}6\,M \\ \text{baixo}, & \text{caso contrário.} \end{cases}$$

7.3 Entrada

Soma a quantidade ao saldo do material no depósito escolhido (padrão: Depósito A), criando o registro de estoque caso ele ainda não exista. Com `novoMaterial`, o material é cadastrado na mesma operação (nome e categoria obrigatórios). A data informada só é aceita no formato AAAA-MM-DD; caso contrário, vale a data atual no fuso de Recife.

7.4 Saída

Exige obra de destino e quantidade positiva. O sistema retira primeiro dos depósitos com maior saldo, até completar a quantidade, ou apenas do depósito indicado em `local`. Se o total disponível for menor que o solicitado, a operação é recusada e a mensagem informa o saldo disponível. O responsável é o usuário logado, salvo se outro nome for informado.

7.5 Transferência

Exige origem e destino válidos e diferentes, e saldo suficiente na origem. A quantidade é subtraída da origem e somada ao destino na mesma chamada de gravação.

7.6 Histórico

A consulta é filtrada e paginada no servidor (8 itens por página por padrão, máximo de 100) e ordenada da mais recente para a mais antiga. A busca de texto cobre material, código e responsável, e caracteres especiais são escapados. A resposta inclui o total de registros por tipo (resumo), usado nos cartões da tela.

7.7 Verificação de e-mail

- No cadastro, o usuário é criado como não verificado e recebe um código de 6 dígitos, válido por 30 minutos.
- São permitidas até 5 tentativas erradas por código; depois disso o código deixa de funcionar e é preciso pedir outro.
- Cada código só vale uma vez. Ao gerar um novo, os anteriores do mesmo tipo são invalidados.
- O reenvio só ocorre se o último envio tiver mais de 60 segundos.
- Um novo cadastro com e-mail ainda não verificado atualiza os dados da conta pendente e reenvia o código, em vez de rejeitar.
- Falha no envio do e-mail não impede o cadastro: a resposta traz `emailEnviado = false`.

7.8 Recuperação de senha

O fluxo usa o mesmo mecanismo, com código válido por 15 minutos. A solicitação sempre responde `ok`, exista ou não a conta, para não revelar quais e-mails estão cadastrados. `verificarCodigoReset` apenas confere o código, sem consumi-lo; `redefinirSenha` exige senha com no mínimo 6 caracteres, consome o código e, se a conta ainda não estava verificada, marca o e-mail como verificado.

7.9 Planta dos depósitos

Regra	Valor
Depósitos por planta	de 1 a 50
Nome do depósito	obrigatório, até 40 caracteres, sem repetição (ignora maiúsculas)
Tamanho	largura de 650 a 4000, altura de 490 a 1800 (valores inválidos viram o mínimo)
Cavaletes por depósito	até 500
Identificador do cavalete	inteiro positivo, sem repetição (inválidos e repetidos são descartados)
Nome personalizado	até 20 caracteres
Anotações	até 2000 caracteres
Coordenadas	arredondadas a 2 casas, nunca negativas
Último depósito	não pode ser excluído

8 Autenticação e segurança

- O cadastro usa o e-mail como `username`. Senhas têm mínimo de 6 caracteres e são armazenadas com hash pelo Parse.
- O login só é concluído para e-mails verificados. As funções de dados também rejeitam usuários não verificados.
- O cliente não consegue marcar a própria conta como verificada: o `beforeSave` bloqueia a alteração fora da master key.
- Os códigos de e-mail são guardados apenas como HMAC-SHA256, calculado com o segredo `TOKEN_SECRET`, e comparados em tempo constante.
- Mensagens de erro para códigos inexistentes, expirados ou errados são idênticas, e a recuperação de senha não revela se o e-mail existe.
- Após o login, o servidor devolve um token de sessão, guardado no `localStorage` e enviado no cabeçalho `X-Parse-Session-Token`. Ao abrir o sistema, a função `me` restaura a sessão.
- Se o token expirar (código 209), o front-end o remove e recarrega a página, voltando à tela de login.
- As chaves `Application-Id` e `JavaScript-Key` são públicas por natureza e ficam no front-end. A **Master Key** nunca deve ser colocada no front-end; ela é usada apenas dentro do Cloud Code.
- As Class Level Permissions das classes de dados devem negar acesso público, para que as tabelas só sejam acessadas pelas Cloud Functions.
- `seed` e `testarEmail` exigem a master key. `salvarLead` e as funções de conta (cadastro, verificação, login e recuperação) são as únicas abertas ao público.
- Textos de usuários inseridos nos e-mails são escapados contra HTML.

9 Integração do front-end

O arquivo `app/api.js` centraliza as chamadas. Ele monta a requisição, anexa o token de sessão quando existe e converte erros do Parse em exceções com o campo `campos`.

```
const BASE = process.env.NEXT_PUBLIC_PARSE_SERVER_URL
  || "https://parseapi.back4app.com";

export async function api(fn, params = {}) {
  const token = localStorage.getItem("token");
  const res = await fetch(`${BASE}/functions/${fn}`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "X-Parse-Application-Id": process.env.NEXT_PUBLIC_PARSE_APP_ID,
      "X-Parse-Javascript-Key": process.env.NEXT_PUBLIC_PARSE_JS_KEY,
      ...(token && { "X-Parse-Session-Token": token }),
    },
    body: JSON.stringify(params),
  });
  const data = await res.json();
  if (!res.ok) { /* converte data.error em Error com .campos */ }
  return data.result;
}
```

Exemplo de uso em uma tela (registro de entrada):

```
try {
  await api("registrarEntrada", { codigo, quantidade, local, nota, data, obs })
    ;
} catch (err) {
  setErrors(err.campos || { quantidade: err.message });
}
```

Mudança no fluxo de cadastro. `register` não devolve mais token. O front-end deve exibir a tela de código, chamar `verificarEmail` e só então chamar `login`. Se `login` responder com `codigo = EMAIL_NAO_VERIFICADO`, a tela de código deve ser exibida.

10 Instalação e implantação

10.1 Back-end (Back4App)

1. Criar um aplicativo no Back4App.
2. Em *Cloud Code* → *Functions*, enviar `main.js` como `cloud/main.js` e clicar em *Deploy*.
3. Configurar as variáveis de ambiente do aplicativo (Tabela 6).
4. Executar a função `testarEmail` com a master key para conferir o envio de e-mails.
5. Em cada classe de dados, remover o acesso público nas Class Level Permissions.

A carga inicial demonstrativa (`seed`) está desativada nesta versão: os materiais são cadastrados pelo sistema. A planta cria os depósitos A, B e C na primeira consulta, e a classe `Deposito` é opcional (define apenas a capacidade).

Tabela 6: Variáveis de ambiente do back-end.

Variável	**Descrição**
`EMAIL_PROVIDER`	`brevo` (padrão) ou `resend`
`EMAIL_API_KEY`	chave de API do provedor (obrigatória para enviar e-mails)
`EMAIL_FROM`	endereço do remetente (obrigatório)
`EMAIL_FROM_NAME`	nome do remetente (padrão: Marmopedras)
`ADMIN_EMAIL`	recebe o aviso de novo cadastro (opcional)
`TOKEN_SECRET`	segredo do HMAC dos códigos; deve ser definido com um valor longo e aleatório

10.2 Front-end

Criar o arquivo `.env.local` a partir de `.env.example`:

```
NEXT_PUBLIC_PARSE_APP_ID=seu_application_id
NEXT_PUBLIC_PARSE_JS_KEY=sua_javascript_key
NEXT_PUBLIC_PARSE_SERVER_URL=https://parseapi.back4app.com
```

Em seguida:

```
npm install
npm run dev     # desenvolvimento em http://localhost:3000
npm run build   # build de producao
```

Após alterar o `.env.local`, o servidor de desenvolvimento deve ser reiniciado.

10.3 Problemas comuns

Sintoma	**Causa provável**
Resposta 403 / unauthorized	`APP_ID` ou `JS_KEY` incorretos no `.env.local`
Invalid function	`main.js` não foi publicado com Deploy
Listas vazias	ainda não há materiais ou movimentações cadastrados (a carga `seed` está desativada)
Erro de rede	`NEXT_PUBLIC_PARSE_SERVER_URL` incorreta
"E-mail ainda não verificado" ao entrar	conta sem confirmação; usar o código enviado ou pedir reenvio
Código não chega por e-mail	variáveis `EMAIL_*` ausentes ou incorretas; rodar `testarEmail`
Cadastro responde `emailEnviado = false`	falha do provedor de e-mail; o usuário pode pedir reenvio

Sintoma	Causa provável
"A unidade do material precisa ser atualizada"	material antigo com unidade diferente de "chapas"

11 Testes automatizados

A versão anterior não tinha testes e a validação era manual. Esta versão inclui uma suíte automatizada que exercita o `main.js` sem alterá-lo e sem depender do Back4App, da rede ou de um provedor de e-mail real.

11.1 Estratégia

- **Jest 30**, executado com `npm test`.
- **Parse em memória** (`fakeParse.js`): implementa `Parse.Object`, `Parse.Query` (filtros, `or`, ordenação, `skip`, `limit`, `count`), `Parse.User` (com `signUp` e `logIn`, que dispara o `beforeLogin`) e `Parse.Cloud`. Os dados são guardados como cópias e cada consulta devolve objetos novos, de modo que a falta de um `save` no código aparece nos testes.
- **E-mail simulado**: o `fetch` global é substituído por uma função de teste. Os testes leem o corpo de cada e-mail, extraem o código de 6 dígitos e conferem destinatário, assunto, cabeçalhos e provedor.
- **Helpers** (`ctx.js`): `call` (invoca uma função de nuvem), `falha` (espera um erro e devolve seu conteúdo), `makeUser`, `emails` e `ultimoCodigo`.

11.2 Cobertura funcional

Arquivo	Comportamentos verificados
`auth.test.js`	Validação do cadastro; usuário criado como não verificado; código com hash e validade; limite de cinco tentativas; código expirado ou em formato inválido; reenvio com intervalo de 60 s e invalidação do código anterior; recuperação de senha sem revelar se o e-mail existe; troca de senha que consome o código; login com e-mail normalizado; *hooks* `beforeSave` e `afterSave` (inclusive escape de HTML); provedores Brevo e Resend; erro HTTP do provedor; funções restritas à master key.
`estoque.test.js`	Exigência de login em todas as funções de dados; geração sequencial de códigos; duplicidade; mínimo e quantidade inválidos; entrada, saída e transferência com saldos por depósito; saída que consome primeiro o depósito de maior saldo; bloqueio de material com unidade diferente de chapas; faixas de status; filtros, busca, paginação e resumo do histórico; painel; distribuição por depósito; paginação interna com mais de mil registros; leads.

Arquivo	Comportamentos verificados
`planta.test.js`	Criação automática dos depósitos A, B e C; validação de nomes (vazio, longo, duplicado, limite de 50); limites de tamanho; sanitização de cavaletes (identificadores inválidos ou repetidos, arredondamento, truncamento de textos, limite de 500); reordenação e remoção de depósitos; criação e exclusão individuais, com proteção do último depósito.

11.3 Execução e resultado

Na raiz do back-end:

```
npm i -D jest
npm test
npx jest --coverage    # opcional: cobertura de linhas
```

Última execução:

```
 PASS  test/planta.test.js
 PASS  test/auth.test.js
 PASS  test/estoque.test.js

Test Suites: 3 passed, 3 total
Tests:       178 passed, 178 total
Time:        0.408 s
```

11.4 Limites da suíte

- O Parse em memória imita o comportamento do SDK, mas não é o Parse Server real; diferenças sutis podem existir.
- Concorrência não é testada (a execução em memória é sequencial).
- Class Level Permissions, ACLs e o envio real de e-mails dependem do Back4App e do provedor, e não são cobertos.
- O front-end não faz parte desta suíte.

12 Conclusão

O Marmopedras entrega o ciclo completo de controle de estoque (cadastro, consulta, movimentação e histórico), agora com contas protegidas por verificação de e-mail e recuperação de senha, planta dos depósitos e estoque em chapas inteiras. As regras de negócio continuam centralizadas no servidor, em um único arquivo de Cloud Code, e passam a contar com uma suíte de 178 testes automatizados que serve de rede de segurança para novas alterações. Os próximos passos são perfis de acesso, operações atômicas de estoque e a integração real com o WhatsApp.